

---

# ASSIGNMENT #2

## Local Search Algorithms & Genetic Algorithms

|                            |                                  |
|----------------------------|----------------------------------|
| <b>Instructor:</b>         | Miss. Hajra Waheed               |
| <b>Teaching Assistant:</b> | Bilal Ahmad                      |
| <b>Course:</b>             | Artificial Intelligence (CS)     |
| <b>Section:</b>            | 6A                               |
| <b>Total Marks:</b>        | <b>100</b>                       |
| <b>Release Date:</b>       | March 6, 2026                    |
| <b>Due Date:</b>           | <b>March 31, 2026 — 11:59 PM</b> |

### ★ Important Instructions

1. This assignment carries **100 marks** and must be submitted individually.
2. All written solutions must be typed in  $\text{\LaTeX}$ . Code must be included verbatim in the report *and* pushed to your GitHub repository with a clear folder structure (`/local_search/`, `/genetic_algo/`, `/real_world/`).
3. **Show all working steps.** For coding questions, include the complete code listing *and* its full printed output in your report. Code without output receives **zero** marks.
4. All Python code must run without errors. Use only Python's standard library (`random`, `math`, `collections`) — no `numpy`, `scipy`, or `sklearn`.
5. Every coding part also requires written analysis. Code alone without explanation will not receive full marks.
6. **Marks per Sub-part:** To secure full marks in each part, ensure you address *every* requirement listed in that part. Each bullet point or named criterion within a question is a separate marking component — incomplete responses will receive partial marks only.
7. **Academic Integrity:** Plagiarism or use of AI tools to generate solutions is strictly prohibited and will result in a grade of zero and disciplinary action.
8. Submit via Google Classroom before the deadline. **No late submissions will be accepted.**

| Section             | Topic                   | Marks      |
|---------------------|-------------------------|------------|
| Section A (Q1 – Q3) | Local Search Algorithms | 40         |
| Section B (Q4 – Q5) | Genetic Algorithms      | 35         |
| Section C (Q6)      | Real-World Application  | 25         |
| <b>Total</b>        |                         | <b>100</b> |

## Section A: Local Search Algorithms

40 Marks

Questions in this section require you to **implement Hill Climbing variants in Python** and analyse their behaviour experimentally. Submit all code both in your PDF report *and* on GitHub under `/local_search/`. Written analysis is mandatory alongside every coding task — code without explanation receives **zero** marks.

## Q1. Implementing First-Choice &amp; Stochastic Hill Climbing

[14 Marks]

The objective function is defined over integer states  $s \in \{1, 2, \dots, 12\}$  as follows. Each state's neighbourhood consists of its immediate left and right adjacent states only.

| State  | 1 | 2 | 3 | 4  | 5 | 6  | 7  | 8 | 9  | 10 | 11 | 12 |
|--------|---|---|---|----|---|----|----|---|----|----|----|----|
| $f(s)$ | 4 | 9 | 6 | 11 | 8 | 15 | 10 | 7 | 13 | 5  | 16 | 12 |

- (a) [5 marks] Implement **both** First-Choice HC and Stochastic HC in Python as two separate functions with the following interface:

```
first_choice_hc(landscape, start) -> (path, terminal_state)
stochastic_hc(landscape, start) -> (path, terminal_state)
```

Requirements:

- The `landscape` parameter is a Python list where `landscape[i]` is  $f(i + 1)$ .
- Both functions must return the **complete path** (list of states visited) and the final terminal state.
- First-Choice: always check the left neighbour before the right; move immediately on the first improvement.
- Stochastic: collect all strictly uphill neighbours, then pick one using `random.choice()`.
- Write a `main()` block that calls both functions from **every** starting state  $s \in \{1, \dots, 12\}$  and prints a formatted results table with columns: *Start, Algorithm, Path, Terminal State, Steps*.
- No external libraries are permitted beyond Python's built-in `random` module.

- (b) [4 marks] Paste the full output of your `main()` in your report, then answer:

- How many starting states lead each algorithm to the **global maximum** (state 11)? Present this as a summary table.
- Identify **all starting states** where First-Choice and Stochastic HC reach *different* terminal states. For each such state, explain the divergence: which  $f$ -values caused the two algorithms to make different decisions, and why?
- Run Stochastic HC **50 times** from  $s = 4$  in a loop (`random.seed()` must not be fixed — allow true randomness). Record how often it reaches state 11 vs. any local maximum. Report the percentage and write 2–3 sentences on what this reveals about Stochastic HC's reliability.

- (c) [5 marks] Modify your code to handle **plateaus**: change states  $\{5, 6, 7\}$  to all have  $f = 15$  in the landscape.

- Add plateau detection to both functions: print a warning message each time the algorithm finds *no strictly better* neighbour but *at least one equal* neighbour.
- Re-run both algorithms from all 12 starting states. Report how many runs get stuck on the plateau.
- Now implement a **sideways-move** extension: allow movement to equal-valued neighbours, but cap sideways moves at **10 per run**. Re-run and compare success rates before and after this fix.
- Write 4–5 sentences: does the sideways-move fix work equally well for both variants? Which variant benefits more, and why does the cap of 10 matter?

## Q2. Implementing Random Restart Hill Climbing

[14 Marks]

**Scenario:** A warehouse robot is searching for the optimal shelf placement across a storage floor modelled as a 1D landscape of 14 positions. Each position has a utility score based on proximity to packing stations. The landscape contains multiple local optima that trap basic Hill Climbing.

| State | 1 | 2 | 3 | 4  | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |
|-------|---|---|---|----|---|---|----|----|----|----|----|----|----|----|
| f(s)  | 5 | 8 | 6 | 12 | 9 | 7 | 17 | 14 | 10 | 6  | 19 | 15 | 11 | 8  |

(a) [4 marks] Implement **Random Restart Hill Climbing** in Python. Requirements:

- Function signature: `random_restart_hc(landscape, num_restarts, variant='first_choice')`  
Returns (`best_state`, `best_value`, `all_results`) where `all_results` is a list of (`start`, `terminal`, `path`) tuples.
- On each restart, select start state uniformly at random via `random.randint(0, len(landscape)-1)`.
- Support both 'first\_choice' and 'stochastic' via the `variant` parameter — reuse your Q1 implementations.
- Write a `main()` that runs RRHC with `num_restarts=20` under both variants and prints a restart-by-restart table: restart number, start state, terminal state, f-value, global max found (Yes/No).
- Also write a helper `find_local_maxima(landscape)` that returns all positions whose f-value exceeds both neighbours. Print all local maxima for the Q2 landscape.

(b) [5 marks] Conduct the following experiments and include all printed/plotted output in your report:

- Run RRHC with restart counts  $n \in \{1, 3, 5, 10, 20\}$ . For each  $n$ , repeat 100 independent trials and compute the **empirical probability** of finding the global maximum (state 11). Report all five probabilities in a table.
- Compute the **theoretical probability**  $P = 1 - (1 - p)^n$  for each  $n$ , where  $p$  is the fraction of starting states that reach state 11 under First-Choice HC (derive  $p$  from your Q1 output). Show all calculations.
- Write 3–4 sentences comparing your empirical and theoretical values. Where do they agree and where do they diverge? What does any gap tell you about the assumptions behind the formula?

(c) [5 marks] Extend your RRHC code to investigate plateaus:

- Modify the landscape so states  $\{7, 8\}$  both have  $f = 17$ . Re-run RRHC with 20 restarts and report the global maximum discovery rate before and after the plateau modification (use a table).
- Inside your RRHC function, add a **plateau counter**: count how many restarts terminate *on* the plateau (at state 7 or 8) rather than the true global maximum. Print this count alongside your results.
- Write 3–4 sentences: does running more restarts eventually overcome the plateau? Justify your answer with the experimental numbers, not just theory.

### Q3. Diagnosing & Fixing Hill Climbing Failures in Python

[12 Marks]

(a) [4 marks] Write a Python function `diagnose_hc(landscape, start)` that runs First-Choice HC and automatically detects and reports which failure mode was encountered. The function must:

- Detect and label **three failure modes**: (1) *local maximum* — no neighbour is better or equal; (2) *plateau* — no strictly better neighbour but at least one equal; (3) *ridge* — the algorithm oscillates without progress (detect by checking if the same state is visited twice).
- Print a diagnosis message: "Terminated at state X with f=Y. Failure mode: [name]".
- Construct **three small landscapes of your own** (each with at least 5 states, not reusing Q1/Q2) that each trigger exactly one failure mode. Run `diagnose_hc` on each and include the output in your report.
- For each failure mode: write 2 sentences explaining precisely why the HC algorithm cannot recover from it without external modification.

(b) [4 marks] Implement the **N-Queens problem** ( $N = 8$ ) and attempt to solve it using Stochastic HC with Random Restarts:

- Represent the board as a list of 8 integers where `board[i]` is the row of the queen in column  $i$ .
- Define a fitness function `count_conflicts(board)` that returns the number of *attacking pairs* of queens (lower is better; 0 is a solution).
- Implement `stochastic_hc_nqueens(board)` that performs Stochastic HC by randomly swapping two queens' rows if the swap reduces conflicts.
- Implement `solve_nqueens_rrhc(num_restarts)` that applies Random Restart HC until a solution is found or restarts are exhausted.
- Run `solve_nqueens_rrhc(100)`. Report: how many restarts were needed, the final board state, and print the board visually (use Q and . characters). Include this output in your report.

(c) [4 marks] Benchmark your N-Queens solver and write an analytical report:

- Run `solve_nqueens_rrhc(k)` for  $k \in \{5, 10, 25, 50, 100\}$ . For each  $k$ , repeat 30 independent trials. Record: success rate (fraction of trials that find a solution), and average number of restarts needed when successful.
- Present your results in a table with columns:  $k$ , Success Rate, Avg. Restarts to Solution.

- Write 5–6 sentences of analysis: identify the two specific reasons (beyond just saying “local optima”) why Stochastic HC struggles with N-Queens, referencing the **structure of the conflict landscape**. Explain how your experimental results confirm these reasons. State one remaining limitation that even Random Restart HC cannot overcome for  $N = 1000$  and name a more powerful algorithm.

## Section B: Genetic Algorithms

35 Marks

Questions in this section require you to **implement a full Genetic Algorithm in Python** from scratch. Submit all code both in your PDF report *and* on GitHub under `/genetic_algo/`. Fitness curves, output tables, and written analysis are all required alongside the code.

### Q4. Implementing a Full Genetic Algorithm from Scratch

[20 Marks]

You will implement a complete Genetic Algorithm in Python to maximise the function:

$$f(x) = -x^2 + 14x + 5, \quad x \in \{0, 1, \dots, 15\}$$

Represent  $x$  as a **4-bit binary string**. The true maximum is at  $x = 7$ ,  $f(7) = 54$ .

(a) [5 marks] Implement the following core GA components as individual Python functions:

- `decode(chromosome)` — converts a 4-bit binary list to an integer  $x$ .
- `fitness(chromosome)` — returns  $f(x)$  for the given chromosome.
- `roulette_select(population)` — selects one individual using fitness-proportionate (roulette wheel) selection; must use `random.random()` for the spin.
- `single_point_crossover(parent1, parent2, point)` — returns two offspring chromosomes after crossover at the given bit position.
- `mutate(chromosome, p_m)` — applies bit-flip mutation to each bit independently with probability  $p_m$ .

Test each function in isolation: print the decoded value and fitness for chromosomes `[0,1,1,0]`, `[1,0,0,1]`, `[1,1,0,0]`, `[0,0,1,1]` and include all output in your report.

(b) [5 marks] Build a complete `run_ga(pop_size, num_generations, p_m, elitism)` function that:

- Initialises a random population of `pop_size` chromosomes.
- Runs for `num_generations` generations.
- Each generation: uses roulette selection to pick parents, applies crossover at a random point, then mutation at rate `p_m`.
- If `elitism=True`, carries the best individual from the current generation into the next unchanged.
- Returns a list of `(generation, best_fitness, best_x)` tuples — one per generation.

Run with `pop_size=4, num_generations=10, p_m=0.1, elitism=False`. Print a generation-by-generation table showing: generation, full population chromosomes, decoded x-values, fitness values, and the best individual. Include full output in your report.

(c) [5 marks] Run the following controlled experiments and report all results:

- Run your GA with `pop_size=4`, `num_generations=20`, `p_m=0.1` with `elitism=False` vs `elitism=True`, each repeated 30 times. Report: average best fitness reached, number of runs that found  $x = 7$  (the true optimum), and average number of generations to first reach  $f \geq 50$ .
- Run with `p_m`  $\in \{0.01, 0.1, 0.3, 0.5\}$ , 30 trials each. Report average best fitness per mutation rate in a table.
- Write 4–5 sentences of analysis: how does elitism affect convergence speed vs. solution quality? Which mutation rate performed best and why is a very high  $p_m$  harmful?

(d) [5 marks] Decode and analyse one complete generation manually in your report (no code required for this part):

- Use this exact starting population:  $P_1 = [0, 1, 1, 0]$ ,  $P_2 = [1, 0, 0, 1]$ ,  $P_3 = [1, 1, 0, 0]$ ,  $P_4 = [0, 0, 1, 1]$ .
- Decode all four chromosomes, compute  $f(x)$  for each, and fill in a table with columns: Individual, Chromosome,  $x$ ,  $f(x)$ , Selection Probability.
- Using random numbers  $r_1 = 0.12$ ,  $r_2 = 0.47$ ,  $r_3 = 0.68$ ,  $r_4 = 0.91$ , identify which individuals are selected as Parent Pair 1 and Parent Pair 2.
- Apply single-point crossover at position 2 (after bit 2) and show both offspring for each pair. Decode and evaluate  $f(x)$  for all four offspring.
- Apply bit-flip mutation to offspring  $O_1$  using per-bit random values  $[0.08, 0.43, 0.91, 0.05]$  with  $p_m = 0.1$ . Show the mutated chromosome and its fitness.

## Q5. Implementing a GA for the Course Scheduling Problem

[15 Marks]

**Problem:** Schedule 6 courses  $\{C_1, \dots, C_6\}$  into 4 time slots  $\{T_1, T_2, T_3, T_4\}$  and 3 rooms  $\{R_1, R_2, R_3\}$  such that no two courses share the same room at the same time. Implement a complete GA to solve this in Python.

(a) [4 marks] Implement the GA representation and fitness:

- Represent a chromosome as a Python list of 6 tuples: `[(room_idx, slot_idx), ...]` — one per course. Room index  $\in \{0, 1, 2\}$ , slot index  $\in \{0, 1, 2, 3\}$ .
- Write `random_chromosome()` that generates a random valid-format chromosome.
- Write `count_conflicts(chromosome)` that counts pairs of courses assigned to the same room and same time slot (each such pair = 1 conflict).
- Write `fitness(chromosome)` as  $f = 100 - (10 \times \text{conflicts})$ . A conflict-free schedule scores 100.
- Generate and print 5 random chromosomes, their conflict counts, and fitness values in a table.

(b) [4 marks] Implement constraint-aware crossover and mutation:

- Implement `crossover(p1, p2, point)` — standard single-point crossover that may produce conflicting offspring. Show with two specific example chromosomes (print them) how crossover creates a conflict.
- Implement `repair(chromosome)` — a post-crossover repair function that detects conflicting courses and reassigns them to a random conflict-free (room, slot) combination. If no conflict-free slot exists, assign randomly.

- Implement `mutate(chromosome, p_m)` — with probability `p_m` per gene, reassign that course to a random (room, slot) pair.
- Demonstrate repair: create one conflicting chromosome manually, call `repair()`, and print the before/after conflict counts.

(c) [4 marks] Build and run the full scheduling GA:

- Implement `run_scheduling_ga(pop_size, generations, p_m)` using tournament selection (tournament size = 2): randomly pick 2 individuals, select the fitter one.
- Apply repair after every crossover.
- Track and store the best fitness per generation.
- Run with `pop_size=20, generations=50, p_m=0.1`. Print the best schedule found (all 6 course assignments) and its conflict count. Include all output in your report.
- If a conflict-free solution is found, print: "Solution found at generation X: [chromosome]".

(d) [3 marks] Written analysis (no additional code required):

- Based on your experimental output, did your GA consistently find a conflict-free schedule? If not, explain which constraint makes this hard — reference the ratio of valid schedules to total possible assignments.
- State two conditions under which this GA is *preferable* to Random Restart HC for this scheduling problem, and two conditions under which RRHC would be preferable. Ground your answer in the specific characteristics of the problem (search space size, constraint density, population diversity).

### Section C: Real-World Application

25 Marks

This section requires you to build a **complete working Python system** that applies the course algorithms to a real-world problem. Submit code on GitHub under `/real_world/`. Empirical results, comparison plots, and written analysis are all required — a report without running code receives **zero** marks for coding sub-parts.

## Q6. Building a Real-World Driver Positioning System

[25 Marks]

**Scenario:** A ride-hailing company needs to pre-position exactly **10 drivers** across a  $6 \times 6$  city grid (36 zones) before peak hours to maximise demand coverage. Each zone has a demand score and a supply penalty. No two drivers may share a zone (hard constraint). Your task is to build, run, and compare two complete Python optimisation systems.

The demand grid is defined as follows (row-major order, zones indexed 0–35):

```
demands = [12,45,23,67,34,19, 56,38,72,15,49,61, 27,83,41,55,30,77,
           64,18,52,39,71,26, 44,91,33,58,22,85, 16,69,47,74,31,53]
```

`supply_penalty = 5` (deducted per driver placed in any zone).

Objective: maximise  $\sum_{i \in \text{placed}} \text{demands}[i] - 5 \times 10$ .

(a) [4 marks] Formulate and implement the problem foundation:

- Implement `state_fitness(state, demands)` where `state` is a Python `set` of 10 zone indices. Returns the objective value.

- Implement `get_neighbours(state)` that generates all neighbours by **swapping one driver** from a placed zone to any unoccupied zone. Each neighbour differs from the current state by exactly one zone substitution.
- Implement `random_state()` that generates a valid random initial state (10 unique zone indices sampled without replacement using `random.sample()`).
- Print the fitness of 3 random states and the number of neighbours each has. Verify that your neighbour function always returns states of size 10 with no duplicates.

(b) [6 marks] Implement and run the **Random Restart HC** system:

- Implement `hc_driver(state, demands, variant)` — a single HC run (support both 'first\_choice' and 'stochastic' variants). Returns (`final_state`, `final_fitness`, `steps`).
- Implement `rrhc_driver(num_restarts, demands, variant)` — runs RRHC and returns the best state found across all restarts, plus a list of per-restart best fitness values.
- Choose either First-Choice or Stochastic HC for your restarts. **Justify your choice in writing** (3–4 sentences) referencing the neighbourhood size of this problem.
- Run RRHC with `num_restarts=30`. Print: the best state found, its fitness, which zones the 10 drivers are placed in (use row/column format e.g. (`row=2`, `col=4`)), and the best fitness per restart as a list.

(c) [6 marks] Implement and run the **Genetic Algorithm** system:

- Represent a chromosome as a **sorted list** of 10 unique zone indices (e.g., `[0,5,14,21,26,28,31,33,34,35]`).
- Implement `ga_fitness(chromosome, demands)` — same objective as above.
- Implement `ordered_crossover(p1, p2)` — use Order Crossover (OX): copy a random slice from parent 1, then fill remaining positions with values from parent 2 in their original order, skipping duplicates. This naturally preserves the uniqueness constraint.
- Implement `ga_mutate(chromosome, p_m)` — with probability `p_m`, swap one randomly chosen zone in the chromosome with a randomly chosen zone *not* in the chromosome.
- Implement `run_driver_ga(pop_size, generations, p_m)` using tournament selection (`size=3`). Run with `pop_size=30`, `generations=100`, `p_m=0.1`. Print: best solution found, its fitness, driver zone assignments.

(d) [5 marks] Run a head-to-head comparison and write a structured analysis:

- Run both RRHC (30 restarts) and GA (30 pop, 100 gen) each **20 independent times**. Record the best fitness found per trial for both algorithms.
- Report: mean best fitness, standard deviation, and best single run for each algorithm in a table.
- Write a structured discussion (5–6 sentences, no table) covering: which algorithm found higher-quality solutions on average; which was more consistent (lower std); how the hard constraint (no duplicate zones) is handled differently by the two approaches; and how you would scale each approach if the grid grew to  $10 \times 10$  with 30 drivers.

(e) [4 marks] **Real-World Research:** Find and cite one real published paper or documented industrial case where either Hill Climbing or a Genetic Algorithm was applied to a *vehicle dispatch*, *resource placement*, or *ride-hailing optimisation* problem. Your report entry must address:

- The exact problem solved and the scale of the dataset or environment used
  - The specific algorithm variant applied and how they represented the solution space
  - One key quantitative result or improvement reported
  - How their constraint handling compares to your approach in parts (b) and (c)
- 

**Good Luck!**